



CortexAI Session & Token Management Documentation

This document explains how tokens (Google JWTs) and sessions (Redis & Cookies) are generated, validated, and managed across CortexAI from start to finish.

1. Tokens vs. Sessions: A Beginner's Guide

- **Google ID Token (JWT):**
 - *What it is:* A temporary, securely signed passport issued by Google.
 - *Where it is used:* Used strictly during login to verify the user's identity to the backend.
 - **Session ID (UUID):**
 - *What it is:* A unique random key generated by the Auth Service on successful login.
 - *Where it is used:* Stored in a browser cookie and Redis to identify the user for future requests, removing the need to authenticate with Google on every page view.
-

2. Variables & Functions Dictionary

Token Acquisition & Verification

- `GoogleAuthProvider` (Firebase class):
 - **What it is:** Tells the Firebase client SDK to open a Google login provider window.
- `signInWithPopup` (Firebase Auth function):
 - **What it does:** Launches the Google authentication popup flow.
- `getIdToken` (Firebase function in `home.jsx`):
 - **What it does:** Retrieves the JWT token string from the authenticated Firebase client instance.
 - **Variable:** `token` (stores the raw JWT string).
- `verifyIdToken` (Firebase Admin function in `auth.controller.js`):

- **What it does:** Decrypts and verifies the signature of the incoming Google JWT.
- **Variable:** `decoded` (stores the decrypted payload, including the user's `uid` , `name` , `email` , and avatar picture).

Session Storage & Access

- `sessionId` (UUID variable in `auth.controller.js`):
 - **What it is:** Generated using `crypto.randomUUID()` . Used as a secure session key.
- `session-${sessionId}` (Redis key-value pair):
 - **What it stores:** Holds the stringified user object `{ userid, name, email, avatar }` in the Redis cache.
- `session` (Browser Cookie):
 - **How it works:** Holds the `sessionId` UUID string. It is configured with `httpOnly: true` (XSS security) and a `maxAge` of 7 days.
- `protect` (middleware function in `auth.middleware.js`):
 - **How it works:** Reads the `session` cookie from the incoming request, queries Redis for the session details, and maps them to `req.user` .

Identity Propagation

- `proxyWithHeader` (decorator function in `proxywithheader.js`):
 - **How it works:** Takes the user's MongoDB ID (`req.user.userid`) and forwards it downstream as the `x-user-id` header to other microservices.

3. Global Integration (Where & How Session/Token Data is Consumed)

Session and Token details are passed and verified across multiple files in the workspace:

A. Token Lifecycle (`token`)

- **Generated in** `home.jsx` : Acquired from the Firebase client library following successful Google Popup sign-in.
- **Transmitted via Axios:** Passed from the client to the Gateway's `/auth/login` route in

the request body.

- **Decoded in `auth.controller.js`** : Verified inside the Auth Service by the Firebase Admin SDK to check details like expiration, user email, name, and profile photo before creating a database profile.

B. Session Lifecycle (`sessionid` & Cookie)

- **Created in `auth.controller.js`** : Generated after validating the user.
- **Stored in Redis**: Mapped under `session-${sessionid}` key to hold user details in the fast in-memory database.
- **Browser Cookie**: Set as a cookie named `session` with `httpOnly: true` (protecting it from cross-site scripting vulnerabilities).
- **Read in `auth.middleware.js`** : Handled by the Gateway's `protect` middleware on incoming HTTP requests. The middleware pulls the `sessionid` from request cookies to execute the Redis lookup.
- **Cleared in `auth.controller.js`** : Wiped from both Redis and browser cookies on logouts.

C. Database User ID (`x-user-id` Header)

- **Loaded in Gateway Middleware**: Parsed from the Redis session string and attached to `req.user.userid`.
 - **Injected in Gateway Router**: Picked up by the `proxyWithHeader` decorator utility, which writes the `x-user-id` header to requests proxied to downstream services.
 - **Consumed in Chat Service (`chat.controller.js`)**: Extracted from headers (`req.headers["x-user-id"]`) to link conversations and message logs to the logged-in user.
 - **Consumed in Agent Service (`agent.controller.js`)**: Extracted from headers to identify the owner of the active conversation before forwarding prompts to LangGraph.
-

4. The Lifecycle Workflow (File-to-File)

Below is the step-by-step path detailing how tokens and sessions flow across the files:

Phase A: Token Submission & Session Creation (Login)

```
[Trigger] home.jsx (Frontend)
|   —> User logs in with Google popup.
|   —> Extracts Google JWT via data.user.getIdToken() into "token".
|   —> Posts token to Gateway URL at "/auth/login".
▼
[Route Proxy] gateway/index.js
|   —> Forwards request to Auth Service (Port 8001).
▼
[Verificator] auth.controller.js (Auth Service)
|   —> Verifies token using admin.auth().verifyIdToken(token) inside firebase.js.
|   —> Finds or registers User inside MongoDB using user.model.js.
|   —> Generates a secure session ID UUID: "sessionid".
|   —> Caches session details in Redis: redis.set("session-" + sessionid, userJSON, "EX", 3600).
|   —> Attaches "session" cookie with httpOnly: true to HTTP headers.
|   —> Sends 200 OK response back to the client browser.
```

Phase B: Session Verification (API Requests)

```
[Trigger] App.jsx (Frontend)
|   —> On app load/refresh, calls getCurrentuser() features file.
|   —> Sends HTTP GET request to Gateway "/me" (browser automatically carries the session cookie).
▼
[Security Guard] auth.middleware.js (Gateway)
|   —> Intercepts request via protect middleware.
|   —> Reads sessionId from request cookies.
|   —> Checks cache: redis.get("session-" + sessionId).
|   —> If found, parses session JSON and binds user details to req.user.
▼
[Controller] user.controller.js
|   —> Returns req.user profile to the client.
▼
[State Update] App.jsx
|   —> Dispatches user profile to Redux store, updating the UI.
```

Phase C: Logout (Invalidation)

[Trigger] `logout.js` (Frontend)

| —> Sends request to Gateway `/auth/logout` (carrying session cookie).



[Deletor] `auth.controller.js` (Auth Service)

| —> Reads `sessionId` from cookies.

| —> Wipes session key from Redis: `redis.del("session-" + sessionId)`.

| —> Clears cookie: `res.clearCookie("session")`.

| —> Client resets Redux store user state to null.